

Programmazione Dispositivi Mobili

Descrizione del progetto finale

Nome del progetto di laboratorio:

Parkpal



**UNIVERSITÀ
DI TORINO**

GRUPPO

Informazioni sul gruppo di laboratorio.

Nome del gruppo:	Bensoftwares
Componenti:	Taha Benslimane
Github/Gitlab Repo Link:	https://github.com/LeBens03/Parkpal2

DATE

Le date principali del documento.

Data di sottomissione della proposta di progetto	17/04/2025
Data di accettazione della proposta di progetto	06/05/2025

DESCRIZIONE BREVE

Descrizione della App in una frase.

ParkPal allows users to save their car's location when parked and helps them navigate back to it.

DEFINIZIONI

Di seguito la definizione dei termini, abbreviazioni e acronimi utilizzati.

Termine	Definizione

1. CONTESTO DEL PROGETTO

Lunghezza suggerita: 2 pagine

1.1. Situazione attuale

Many apps like ParkPal already exist in the Play Store. Allowing to save the location of your car and navigate back to it. Some offer some additional features like location sharing, taking notes and photos. However, while going through the reviews of these apps, most of them are about the fluidity of the UI/UX, etc. Also, very few of them allow parking different vehicles.

1.2. Benefici e creazione di valore

Parkpal focuses on a simplistic UI with a Google SignIn and a very intuitive interaction for the user. On top of the main features which are the save the user's parking location and help him navigate back to it, the user can register multiple vehicles and choose which one to park. Moreover, the user can access a history of his past parking locations and share the his parking location with other people.

1.3. Obiettivi del progetto

Elenco di aspetti specifici e verificabili che realizzerete nella vostra App e che la renderanno “appetibile”.

UI & Design Layout	The app benefits of simple minimalist design and a fluid user experience. It contains a bottom bar navigation, a modern swipe to delete is used for deletable objects, icons are used to highlight the function of some activities. The app supports a dark mode as well. Jetpack Compose is used for the UI as well as Material Design, with State hoisted in the ViewModels to ensure the UI reacts dynamically to state changes.
Authentication	The app allows both an email + password SignIn using Firebase and a Google SignIn
Data Storage	For data storage, the app uses Room. It's accessed through a DAO class, that manage queries, which in turn is injected in a repository.
Application Architecture	A MVVM architecture is used. The repositories are injected in the view models, used Dagger Hilt. The ViewModels methods are invoked by the UI screens upon user interaction. Also, a clean architecture was applied, the models used by the ui are separate from the entities of the database. Mappers are used to convert from one to the other.
Map & Navigation	The map is powered by Google Maps using it's SDK, and the style of the map is taken from a website called SnazzyMaps that provides different map styles. The app uses an intent to redirect the user to Google Maps for navigation back to the parked car location and allow location sharing to other people.

2. PROFILO DEL PROGETTO

Lunghezza suggerita: 1 pagina

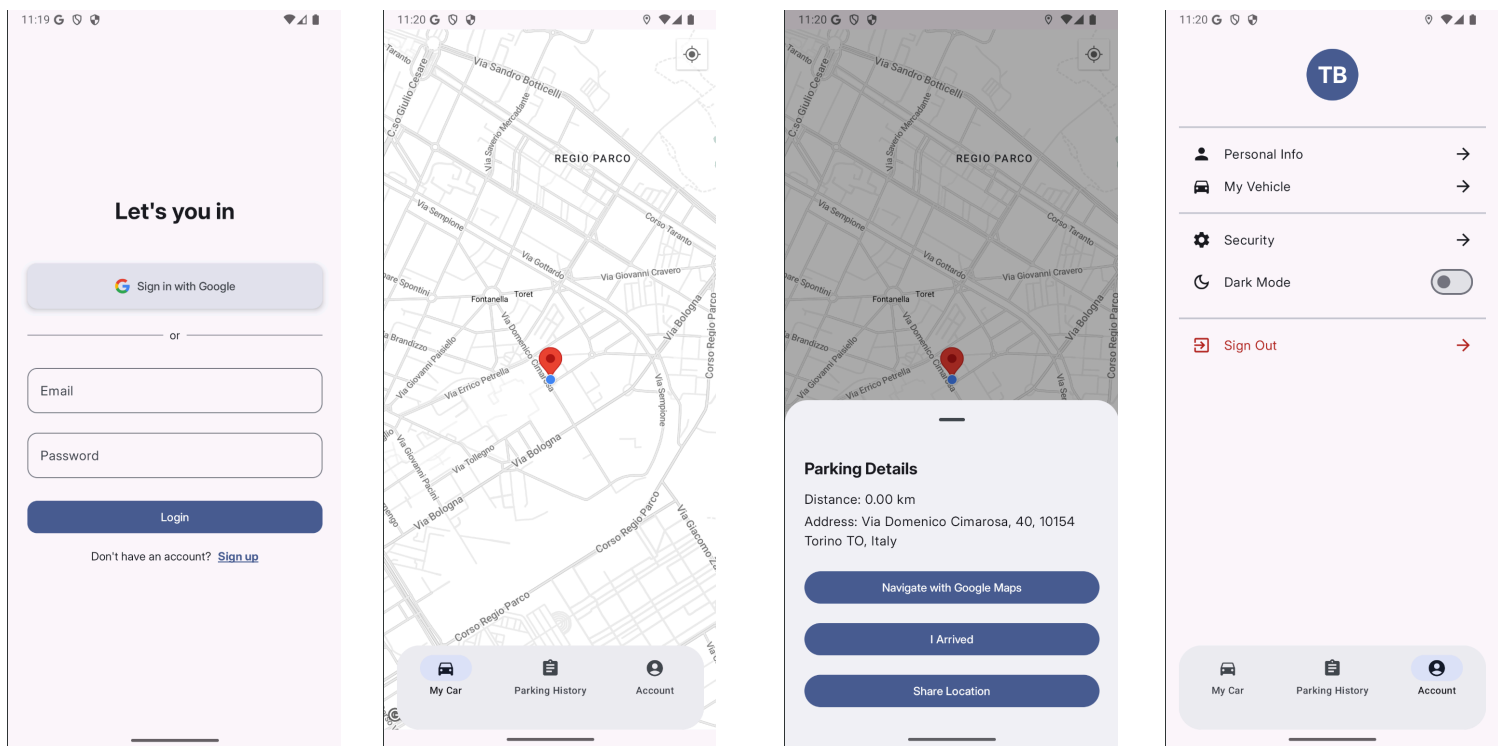
2.1. Ambito del progetto

Parkpal's main feature is to save a user's parking location and helps him navigate back to it. It allows him to share his location and can access a history of his past parking locations. It supports registering multiple cars with the user's account.

For the moment, the app redirects to Google Maps for navigating back to the parking location. When the user arrives he clicks on an "I arrived" button and the parking location is saved in the history. What I'd like to implement in the future, is an in-app navigation instead of a redirection to Google Maps. I'd also like to add a feature that allows the user to save his favorite parking spots and allow him to take pictures of the location as well as note taking when saving a parking location. To go a step further, I'd like to establish a Bluetooth connection to the car, and automatically save the car's location as soon as the user surpasses a certain distance threshold from his car.

Regarding external services, the app interacts with Google Maps for the map display and navigation, and Firebase for the user authentication.

2.2. Profilo della soluzione



3. VINCOLI E ASSUNTI (REQUIREMENTS)

Lunghezza suggerita: 2 pagine

3.1. Vincoli tecnologici

The ParkPal application is developed for the Android platform using the following technologies:

- **Language & Framework:** Kotlin with Jetpack Compose
- **Architecture:** MVVM (Model-View-ViewModel)
- **Database:** Room (for local storage)
- **Maps & Navigation:** Google Maps SDK
- **Asynchronous tasks:** Kotlin Coroutines
- **Authentication:** Firebase

Functional Requirements

- The user must be able to:
 - Sign in using an email and password or through Google
 - Choose which of his registered vehicles he is using
 - Save their current parking location.
 - View the saved location on an embedded map.
 - Launch navigation to the parking spot using Google Maps.
 - Share the parking location with other people through Mail, Whatsapp, etc.
 - Visualize his past parking locations
 - Update his personal information
 - Delete or add vehicles
 - Change his login credentials

- The app should:
 - Handle basic location permissions
 - Be intuitive and easy to navigate. On back button click, it should navigates back to the previous screen
 - Support dark mode based systems settings or user preferences
 - Persist user data on after the app is closed or if the device is restarted
 - Follow best practices of modularity and clean architecture
 - Securely store and authenticate user credentials
 - Follow a clean, modular MVVM architecture

3.2. Eventuali assunti

- Location services are enabled and permissions are granted by the user
- The user has an active internet connection to navigate back to his parked vehicle location and/or share it's location with other people
- The device runs on Android 9.0 (Pie) or higher

4. DESIGN

Lunghezza suggerita: 2-3 pagine

ParkPal follows a **clean, modular MVVM (Model-View-ViewModel)** architecture to ensure scalability, maintainability, and separation of concerns. The application is structured into three key layers: **Presentation, Domain, Data**.

4.1 Architecture Overview

Presentation Layer

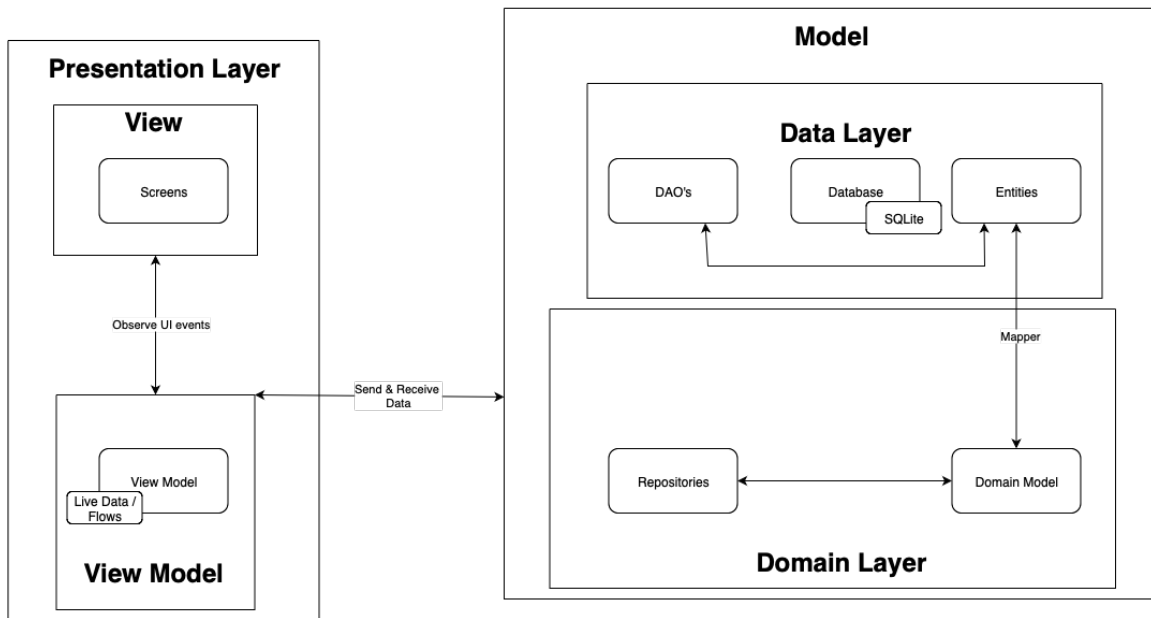
- Composed of Jetpack **Compose screens**.
- UI logic is managed by **ViewModels**, which hoist and expose UI state using `LiveData`, `StateFlow`, or `MutableState`.

Domain Layer

- Contains **business models** that abstract away database-specific logic.
- Defines **repository classes** that expose operations used across the app. These classes are injected into ViewModels to keep them decoupled from data sources.

Data Layer

- Implements data persistence using **Room**. It includes:
 - **Entities**: Database tables
 - **DAOs**: Interfaces for accessing data
 - **Mappers**: Convert between Room entities and domain models to keep the domain layer clean and framework-agnostic.
 - The Room database that ties together all entities and DAOs.



4.2 Jetpack Component Usage

- UI & Layout**
 In addition to the basic layout components, `ModalBottomSheet Composable` is used for choosing which vehicle to park, adding a new vehicle, etc. `SwipeToDismissBox composable` is used for deleting registered vehicles by a swipe to the left.
- LiveData, Flows & State:**
 The state of each screen's variables is hoisted in the `ViewModel` and exposed to the UI using `LiveData`, `StateFlow`, or `MutableState`. The UI then observes this state using `collectAsState()` or `observeAsState()` and reacts to updates automatically.
- Coroutines:**
 All heavy operations—such as database access or background computations—are executed within coroutines in the `ViewModel` to prevent blocking the main thread and to ensure a smooth and responsive user experience.
- Room:**
 Local storage for user data, car information, and parking history is managed using `Room`. `DAOs` are injected into repositories, and `Room` entities are mapped to domain models to maintain a clear separation between the data and domain layers.

- **Navigation:**
The app uses Jetpack Navigation Compose to manage screen transitions within a `NavHost`, utilizing `NavController` and `composable` destinations.
- **Intent:**
Intent is used to redirect the user to Google Maps when navigating back to his parking location, as well as to allow the user to share the parking location via external apps like Email, Whatsapp or SMS.

4.3 Architectural Alignment with Requirements

This architecture adheres to the principles of **MVVM** and satisfies the architectural requirements of a modern Android application:

- **Separation of concerns** is clearly maintained:
 - The **ViewModel** layer handles business logic and state.
 - The **Model** layer manages data access and domain models.
 - The **View** layer (UI) observes and reacts to state without holding logic.
- **Lifecycle awareness** is achieved through ViewModels, which survive configuration changes and prevent memory leaks.
- **Modularity** is enforced through the separation into presentation, domain, data, and DI layers, each with distinct responsibilities.
- **Testability** is improved, as ViewModels can be tested independently of the UI and repositories can be mocked via interfaces.

5. IMPLEMENTAZIONE

Lunghezza suggerita: 2-3 pagine

The implementation of ParkPal follows the MVVM architectural design, separating concerns between UI, business logic, and data access.

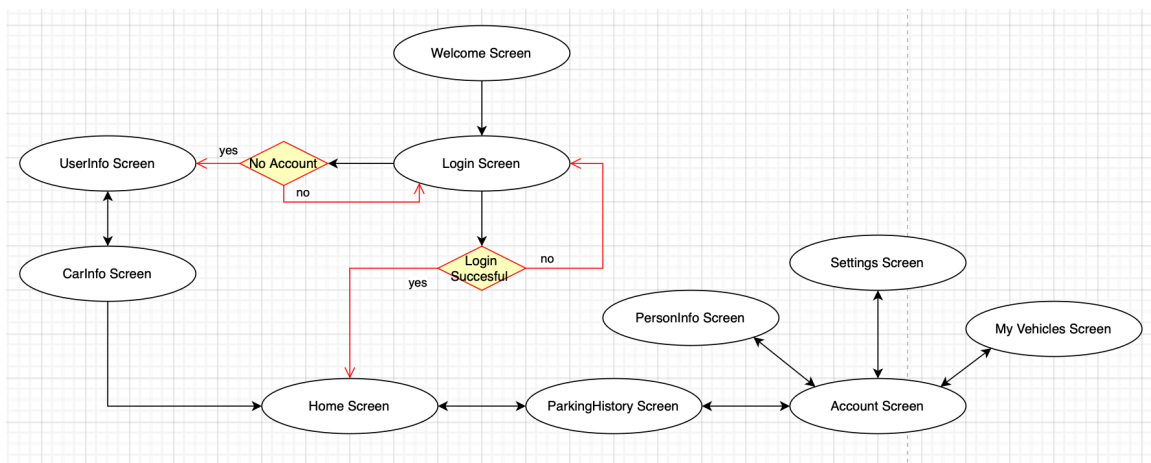
5.1 Class Organization

The source code is organized into the following main packages:

- **presentation:** Contains all **UI screens**, **bottom sheet components**, and **ViewModels**. The screens are further divided into:
 - **onboarding** – for login and first-time user flow.
 - **main** – for the core app screens such as Home, My Vehicles, Settings, Account, Parking History.
- **domain:** Contains **business models** (e.g., **User**, **Car**, **ParkingLocation**) and **repository classes** (e.g., **UserRepository**).
- **data:**
 - **dao** – Room DAOs for database access (**UserDao**, **CarDao**, etc.).
 - **entity** – Room entities corresponding to each domain model (**UserEntity**, **CarEntity**).
 - **mapper** – Responsible for converting between entities and domain models.
 - **AppDatabase.kt** – Defines the Room database with access to DAOs.
- **di:** Dependency injection layer using **Hilt**, including:
 - **DatabaseModule.kt** – Provides Room database and DAOs.
 - **RepositoryModule.kt** – Provides repository instances to ViewModels.

- **theme:** Contains application-wide styles, colors, shapes, and typography for consistent UI theming.
- **utils / location:** Includes utility classes like `UserLocationImpl.kt` and location/state/event files like `MapState.kt` and `MapEvent.kt`.
- **MainActivity.kt & AppNavigation.kt:** Root entry point and navigation controller setup.

5.3 Navigation Graph



5.3 Implementation Details

- **Vehicle selection** and **car addition** are implemented using Jetpack Compose's `ModalBottomSheet` Composable for a smooth UX.
- **Swipe-to-dismiss** actions in the vehicle list use `SwipeToDismissBox` Composable to allow deletion gestures.
- **Location tracking** is implemented via `UserLocationImpl.kt` using `FusedLocationProviderClient`, exposing updates to `MapViewModel`.
- **Google Sign-In** is handled in `GoogleSignInClient.kt`, and integrated with Firebase Auth.
- **Database interactions** are abstracted through repositories using Room, Coroutines, and Hilt-injected DAOs.
- **Dependency injection** is implemented with Hilt to automatically provide DAOs, repositories, and other dependencies to ViewModels and components,
- **Map interaction** logic is organized using `MapEvent.kt` and `MapState.kt`, modeling user interactions and reactive map state.
- **Navigation** is managed using Jetpack Navigation Compose, with a dynamic bottom navigation bar(`BottomNavigationBar.kt`) and a sealed class (`BottomNavDestinations.kt`) to define routes. No route parameters have been used, instead a singleton instance of the ViewModels is instantiated in `AppNavHost` and passed as parameters of the composable along with callback functions that navigate to next destination when called upon .
- **Intents** are used for:
 - Launching Google Maps with directions to the saved parking spot.
 - Sharing the parking location through apps like WhatsApp or Mail.

6. TEST

Lunghezza suggerita: 2 pagine

The ParkPal application was tested through a combination of automated tests and manual testing to ensure its functionality, reliability, and usability.

Automated Testing

Unit tests were developed to validate the core business logic within the ViewModels responsible for managing user, car, and parking history data. These tests cover successful operations as well as failure and edge cases, such as exceptions or invalid inputs. Mockito was used to mock dependencies, and coroutine testing utilities were employed to handle asynchronous code.

An **instrumented test** was implemented for the SignIn screen to simulate user interaction and verify the correct triggering of authentication flows. This test inputs valid credentials, performs clicks on the login and Google Sign-In buttons, and checks that the authentication state is updated appropriately. A fake ViewModel was used to isolate the UI from the backend authentication logic.

Manual Testing

In addition to automated tests, manual testing was performed during development to check UI navigation, interactions with the map and some other features not easily testable.

7. CONCLUSIONI

Lunghezza suggerita: 1-2 pagine

The final implementation of ParkPal successfully meets the initial project requirements outlined in Section 3. The core functionalities — such as user authentication (via email/password and Google Sign-In), multi-vehicle management, parking location saving, navigation redirection, sharing features, and parking history tracking — have all been realized effectively.

The clean, modular MVVM architecture described in the design section allowed for clear separation of concerns, making the app maintainable and scalable. Jetpack Compose was utilized for the UI, Room handled local data storage, and Firebase managed authentication, providing a modern foundation. Navigation between screens was handled through Jetpack Navigation Compose, fulfilling the requirement for intuitive and responsive user experience.

Testing was essential in validating the app's behavior. Unit tests for ViewModels confirmed that business logic and data handling function correctly under various scenarios, including edge cases. The instrumented UI test for the SignIn screen verified that user interactions trigger the appropriate authentication flows and state changes. Manual testing was used to test less flexible and automatically testable features, mainly the user's map interactions, location permissions, and UI navigation flow, ensuring that the app performs behaves as expected.

Due to time constraints, some planned features such as in-app navigation and Bluetooth integration for automatic vehicle location saving, triggered when the app disconnects from the car or the user moves beyond a certain distance, were not implemented. These future implementations will significantly increase the app's convenience and user experience.